Assignment_1_Introduction_to_R

2026-08

Welcome to the first assignment of Foundations of Data Science!

Instructions

- Write your code in the R code chunks provided. Make sure to comment your code.
- When you are finished with your assignment, upload this R Markdown file under the Assignments tab on our class canvas page.
- You can work with your classmates, but remember to write their name in the comments.
- If you are inspired by code you found online make sure you cite the website, comment your code, and make it your own. No copy and pasting.
- IMPORTANT: Explain what your code does in a sentence or two accompanying each answer. I want to know not just if your code works but if you understand WHY it works.

1) Below I have provided a list of random letters and numbers. Please conduct the following tasks on this list:

a) What type of data is each of the "things" in the list? Write these types to a vector called "my_types". Pssssst here is a hint for how you might want to write this code: https://www.geeksforgeeks.org/adding-elements-in-a-vector-in-r-programming-append-method/#.
b) Count how many of each data type appear in your vector.
c) How many numbers in this list are greater than 5?
d) Generate a factor of types from your "my_types" vector.
e) Get the index (or indices) of the number "1" in the lst list.

```
lst <- list(4, 'j', 'Y', 5, 'f', 'K', 8, 'z', 'T', 'c', 0, 'B', 2, 'm', 'X', 9, 'd', 'V',
 ↪  3, 'n', 'G', 6, 'h', 'J', 2, 'p', 'O', 1, 'q', 'R', 7, 's', 'D', 2, 'l', 'N', 0, 'w',
 ↪  'E', 5, 'e', 'U', 4, 'i', 'P', 1, 'a', 'H', 8, 'y', 'S', 3, 'g', 'F', 9, 'b', 'M', 6,
 ↪  'k', 'L', 7)

## What type of data is each of the "things" in the list? I will iterate over my list and
 ↪  add the type of each item to my newly initialized list using a for loop and the
 ↪  class() base R function ##

my_types <- c() # Create my new empty vector

for(i in seq_along(lst)){ # Start a for loop running over each item in my list
  element_type <- class(lst[[i]]) # Find the type of the current element in the loop
  my_types[i] <- element_type # Append the element to the end
}

print(my_types) # Print my new vector
```

```
##  [1] "numeric"   "character" "character" "numeric"   "character" "character"
##  [7] "numeric"   "character" "character" "character" "numeric"   "character"
## [13] "numeric"   "character" "character" "numeric"   "character" "character"
## [19] "numeric"   "character" "character" "numeric"   "character" "character"
## [25] "numeric"   "character" "character" "numeric"   "character" "character"
## [31] "numeric"   "character" "character" "numeric"   "character" "character"
## [37] "numeric"   "character" "character" "numeric"   "character" "character"
## [43] "numeric"   "character" "character" "numeric"   "character" "character"
## [49] "numeric"   "character" "character" "numeric"   "character" "character"
## [55] "numeric"   "character" "character" "numeric"   "character" "character"
## [61] "numeric"
```

```r
## Count how many of each data type appear in your vector? ##

count_types <- table(my_types) # Create a table with the frequency of each unique type in
 ↪  my vector
print(count_types) # Print the frequency table
```

```
## my_types
## character   numeric
##        40        21
```

```r
## How many numbers in this list are greater than 5? ##

num_greater_than_5 <- length(which(sapply(lst, \(x) is.numeric(x) && x > 5))) # Identify
 ↪  vector with indices in the list (lst) where the element evaluates to TRUE if it is
 ↪  greater than 5, then take the length of that vector, I used sapply with a lambda
 ↪  function because initially I just had x > 5 but the char values were causing a lot of
 ↪  warnings

print(paste(c("There are ", num_greater_than_5," numeric values greater than 5 in the
 ↪  list."), collapse = "")) # Print my output
```

```
## [1] "There are 8 numeric values greater than 5 in the list."
```

```r
## Generate a factor of types from your "my_types" vector. ##

my_types_factor <- factor(my_types) # Generate factor from my types vector

print(my_types_factor) # Print factor
```

```
##  [1] numeric   character character numeric   character character numeric
##  [8] character character character numeric   character numeric   character
## [15] character numeric   character character numeric   character character
## [22] numeric   character character numeric   character character numeric
## [29] character character numeric   character character numeric   character
## [36] character numeric   character character numeric   character character
## [43] numeric   character character numeric   character character numeric
## [50] character character numeric   character character numeric   character
## [57] character numeric   character character numeric
## Levels: character numeric
```

```r
## Get the index (or indices) of the number "1" in the lst list. ##

indices_value_1 <- which(sapply(lst, \(x) is.numeric(x) && x == 1)) # Same proces as
 ↪  above for finding values greater than 5, just using the condition "element == 1"
 ↪  intead

print(paste(c("The number 1 is present at indices ", paste(indices_value_1,collapse = ",
 ↪  "),"in the list."), collapse = "")) # Print my vector of indices
```

```
## [1] "The number 1 is present at indices 28, 46 in the list."
```

2) In the "data" folder of this repository there is a file called GSE165252_vst_PERFECT.txt. This is real RNA-seq data from a phase II clinical trial investigating the impact of combined impact PD-L1 inhibitor (atezolizumab) and neoadjuvant chemoradiotherapy on resectable esophageal adenocarcinoma patients. You can find a link to the data on GEO here: https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE165252. You can read about this study here: https://aacrjournals.org/clincancerres/article/27/12/3351/671431/Neoadjuvant-Chemoradiotherapy-Combined-with. Download the GSE165252_vst_PERFECT.txt file to your local machine.

a) Read in this file. Make sure you take into account that the first column contains Ensembl IDs (gene names)!
b) How many samples are in this data set?
c) How many genes are in this data set?
d) Make a histogram of the expression values of a gene of your choosing from this data set.
e) Subset this data to contain only their first three samples and first 2000 genes. Write this subset to a file called "eso_sub.csv".

```r
## Read in this file. Make sure you take into account that the first column contains
 ↪  Ensembl IDs (gene names)! ##

setwd("~/Dartmouth - Foundations of Data Science - R Studio/week_1") # Set my directory

assignment_1_data <- read.table(file = "GSE165252_vst_PERFECT.txt", sep = "\t", header =
 ↪  TRUE, stringsAsFactors = FALSE) # Read in the data, assuming it is tab delimited,
 ↪  which seems to hold up after inspection

gene_ids <- assignment_1_data$X # Set the gene IDs in case needed

## How many samples are in this data set? ##

dimensions_gene_sample_data <- dim(assignment_1_data[,-1]) # Read the dimensions of the
 ↪  genetic data, removing the column with gene names so it isn't counted in the sample
 ↪  count

num_samples <- dimensions_gene_sample_data[2] # Pull the number of samples (aka number of
 ↪  columns in the data since each column is a sample, this lives in the second index of
 ↪  the vector generated by dim())

print(paste(c("There are ",num_samples," samples in the dataset."),collapse = "")) #
 ↪  Print the number of samples
```

```
## [1] "There are 77 samples in the dataset."
```

```r
## How many genes are in this data set? ##

num_genes <- dimensions_gene_sample_data[1] # Pull the number of genes (aka number of
 ↪  rows in the data since each row is a gene, this lives in the first index of the
 ↪  vector generated by dim())

print(paste(c("There are ",num_genes," genes in the dataset."),collapse = "")) # Print
 ↪  the number of genes
```

```
## [1] "There are 62300 genes in the dataset."
```

```r
## Make a histogram of the expression values of a gene of your choosing from this data
 ↪  set. ##

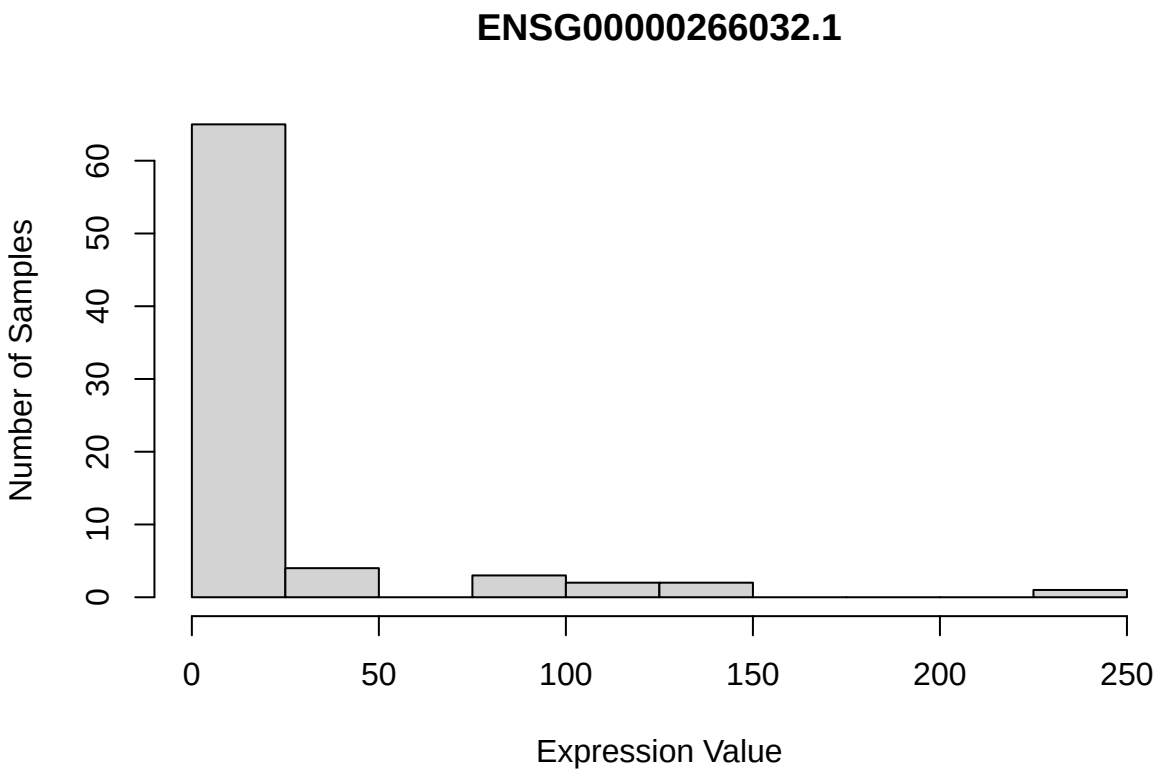
set.seed(123) # Set seed for sampling reproducibility

rand_index <- sample(length(assignment_1_data[,1]), size = 1, replace = FALSE) # Choose
 ↪  an index at random from the length of the data frame column

gene_name <- assignment_1_data[rand_index,1, drop = TRUE] # Pulling the name of the gene
 ↪  we are using

gene_data <- as.numeric(assignment_1_data[rand_index,-1]) # Subset the gene data to the
 ↪  gene of interest and removing the gene name row

hist_expression_values <- hist(gene_data, main = gene_name, xlab = "Expression Value",
 ↪  ylab = "Number of Samples", breaks = seq(0, 250, by = 25)) # Generate histogram based
 ↪  on the gene data for a single gene. Used the smaller breaks for easier interpretation
 ↪  after viewing histogram with breaks at each 50. Labeled the axes and gave it a title
 ↪  so it is easier to view.
```




```
## Subset this data to contain only their first three samples and first 2000 genes. Write
 ↪  this subset to a file called "eso_sub.csv". ##

subset_data <- assignment_1_data[c(1:2000),c(1:4)] # Subset the data to the first 2000
 ↪  genes (first 2000 rows) and first three samples (columns 2-4, retaining column 1 with
 ↪  the gene labels)

write.csv(subset_data, file = "eso_sub.csv") # Write the subsetted data to the csv
```

3) Write a function call "my_first_function". The function should take the following vector: my_vector <- c(1,2,3,4,5,6,7,8,9). Your function should sort the numbers in my_vector into two categories: less than or equal to 5 or greater than 5. Hint: You can take a look at for loops and if/else statements to complete this question!

```
my_first_function <- function(unsorted_vector){ # Define function name
  output_tbl <- data.frame(greater_than_5 = 0, less_than_equal_5 = 0) # Initialize output
 ↪  table locally (doesn't have to be a dataframe this is just what I chose)
  for(i in seq_along(my_vector)){ # Sequence over vector using indices
    if(my_vector[i] > 5){ # Check for the value compared to 5
      output_tbl$greater_than_5[[1]] = output_tbl$greater_than_5[[1]] + 1 # Increment the
 ↪  value in my data frame if needed
    }
    else{ # All other values will be less than or equal to five so no condition (Since
     ↪  our function is designed to take a specific vector with only integers we don't
     ↪  need edge case or error handling)
```

```
      output_tbl$less_than_equal_5[[1]] = output_tbl$less_than_equal_5[[1]] + 1 #
 ↪  Increment the value in my data frame if needed
    }
  }
  return(output_tbl) # Return the output
}

my_vector <- c(1,2,3,4,5,6,7,8,9) # Initialize the vector as requested

categorized_my_vector <- my_first_function(my_vector) # Run my function on the
 ↪  initialized vector

categorized_my_vector # Print out the table
```

```
##   greater_than_5 less_than_equal_5
## 1              4                 5
```